

MOBILE DEVELOPMENT

SERVICES



CONTENTS

ANDROID SERVICE

BROADCAST RECEIVER

EXAMPLES



ANDROID SERVICE

A Service is an application component that runs in the background, not interacting with the user, for an indefinite period of time.

Services, like other application objects (activities, broadcast listeners...), run in the main thread of their hosting process.

This means that, if your service is going to do any CPU intensive (such as MP3 playback) or blocking (such as networking) operations, it should spawn its own thread in which to do that work.

Each service class must have a corresponding <service> declaration in its package's AndroidManifest.xml.

- Services can be started with: **startService()** and **bindService()**.
- Each startService call invokes the **onStart()** method of the service class, **however the service is started only with the first call**.
- Only one **stopService()** call is needed to stop the service, no matter how many times startService() was called.

ANDROID SERVICE

Service Life Cycle

Like an activity, a service has lifecycle methods that you can implement to monitor changes in its state. But they are fewer than the activity methods — only three — and they are public, not protected:

- 1. `void onCreate()`
- 2. `void onStart(Intent intent)`
- 3. `void onDestroy()`



The entire lifetime of a service happens between the time `onCreate()` is called and the time `onDestroy()` returns.

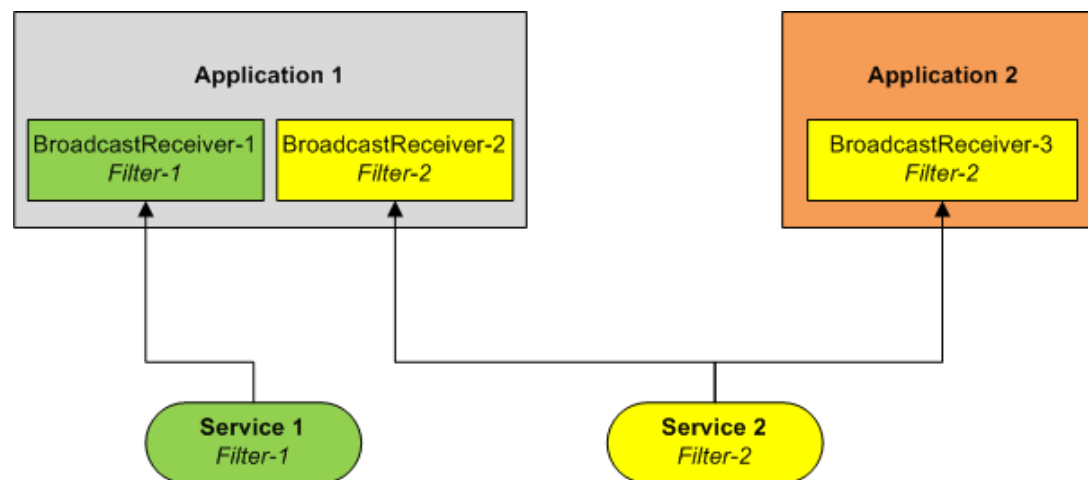
Like an activity, a service does its initial setup in `onCreate()` and releases all remaining resources in `onDestroy()`.

For example, a music playback service could create the thread where the music will be played in `onCreate()`, and then stop the thread in `onDestroy()`.

BROADCAST RECEIVER

A Broadcast Receiver is an application class that listens for global Intents that are broadcasted to anyone who bothers to listen, rather than being sent to a single target application/activity.

The system delivers a broadcast Intent to all interested broadcast receivers, which handle the Intent sequentially.





onReceive

BROADCAST RECEIVER

Registering a Broadcast Receiver

- You can either dynamically register an instance of this class with `registerReceiver()`
- or statically publish an implementation through the `<receiver>` tag in your `AndroidManifest.xml` (see next example).

Broadcast Receiver Lifecycle

void onReceive (Context context, Intent broadcastMsg)

A broadcast receiver has a single callback method:

- 1. When a broadcast message arrives for the receiver, Android calls its `onReceive()` method and passes it the `Intent` object containing the message.
- 2. The broadcast receiver is considered to be active only while it is executing its `onReceive()` method.
- 3. When `onReceive()` returns, it is inactive.

BROADCAST RECEIVER

Services, BroadcastReceivers and the AndroidManifest

The manifest of applications using Android Services must include:

- A <service> entry for each service used in the application.
- If the application defines a BroadcastReceiver as an independent class, it must include a <receiver> clause identifying the component.
- In addition an <intent-filter> entry is needed to declare the actual filter the service and the receiver use.

```
<application android:icon="@drawable/icon" android:label="@string/app_name">
  <activity android:name=".MyServiceDriver2">
    <intent-filter>
      <action android:name="android.intent.action.MAIN" />
      <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
  </activity>
  <service android:name="MyService2"/>
  <receiver android:name="MyBroadcastReceiver"><intent-filter>
    <action android:name="matos.action.GOSERVICE2"/></intent-filter></receiver>
</application>
```

BROADCAST RECEIVER

Types of Broadcasts

There are 2 major classes of broadcasts that can be received:

- 1. Normal broadcasts (sent with `sendBroadcast`) are completely asynchronous. All receivers of the broadcast are run in an undefined order, often at the same time.
- 2. Ordered broadcasts (sent with `sendOrderedBroadcast`) are delivered to one receiver at a time. As each receiver executes in turn, it can propagate a result to the next receiver, or it can completely abort the broadcast (`abortBroadcast()`) so that it won't be passed to other receivers.
 - Ordering receivers for execution can be controlled with the `android:priority` attribute of the matching intent-filter;
 - Receivers with the same priority will be run in an randomized order.

BROADCAST RECEIVER EXAMPLE

(Main steps – main activity)

Assume main activity MyService3Driver wants to interact with a service called MyService3. The main activity is responsible for the following tasks:

1. Start the service called MyService3.

```
Intent intentMyService = new Intent(this, MyService3.class);  
ComponentName service = startService(intentMyService);
```

2. Define corresponding receiver's filter and register local receiver

```
IntentFilter mainFilter = new IntentFilter("GOSERVICE3");  
BroadcastReceiver receiver = new MyMainLocalReceiver();  
registerReceiver(receiver, mainFilter);
```

3. Implement local receiver and override its main method

```
public void onReceive(Context localContext, Intent callerIntent)
```

EXAMPLES

(Main steps – the service)

The Service uses its onStart method to do the following:

1. Create an Intent with appropriate broadcast filter (any number of receivers could match it).

```
Intent myFilteredResponse = new Intent("GOSERVICE3");
```

2. Prepare the extra data ('myServiceData') to be sent with the intent to the receiver(s)

```
Object msg = some user data goes here;  
myFilteredResponse.putExtra("myServiceData", msg);
```

3. Release the intent to all receivers matching the filter

```
sendBroadcast(myFilteredResponse);
```



EXAMPLES

(Main steps – main activity)

The main activity is responsible for cleanly terminating the service. Do the following

1. Assume intentMyService is the original Intent used to start the service. Calling the termination of the service is accomplished by the method

```
stopService(new Intent(intentMyService));
```

2. Use the service's onDestroy method to assure that all of its running threads are terminated, and the receiver is unregistered.

```
unregisterReceiver(receiver);
```



EXAMPLES 1

The main application starts a service. The service prints lines on the LogCat until the main activity stops the service. No IPC occurs in the example.

```
class TestMyService1 : AppCompatActivity() {  
    lateinit var intentMyService1 : Intent; lateinit var service: ComponentName  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState); setContentView(R.layout.activity_main)  
  
        val txtMsg : TextView = findViewById(R.id.txtMsg); txtMsg.setText("MyService1 started\n (see LogCat)")  
        intentMyService1 = Intent(this, MyService1::class.java)  
        service = startService(intentMyService1)!!  
  
        val btnStopService : Button = findViewById(R.id.btnStopService)  
        btnStopService.setOnClickListener {  
            stopService(intentMyService1); txtMsg.setText("After stopping Service: \n" + service.getClassName())  
        }  
    }  
}
```

EXAMPLES 1

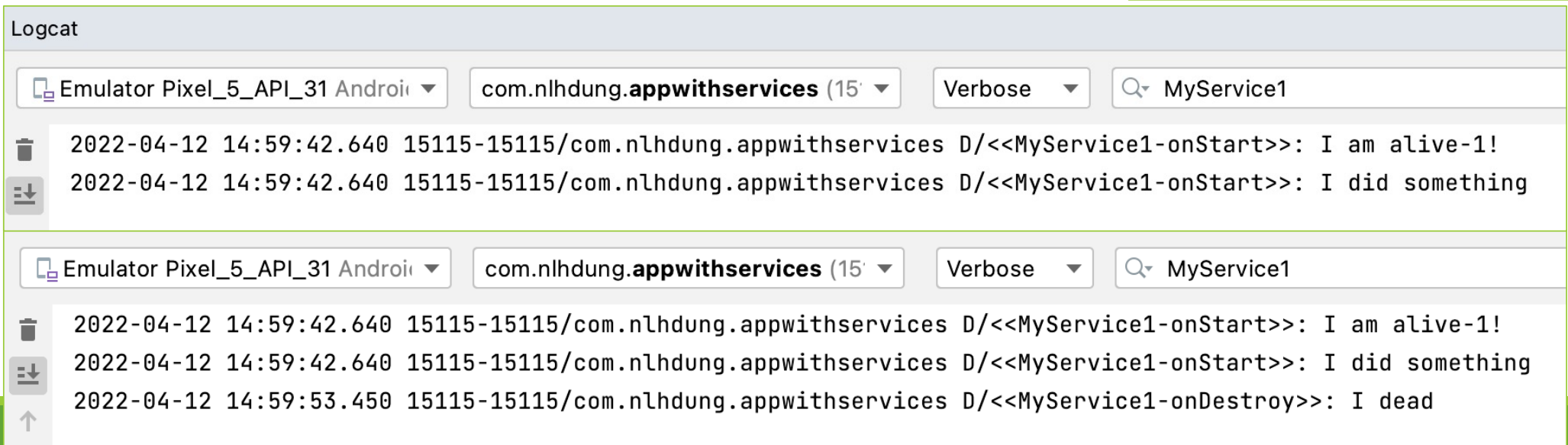
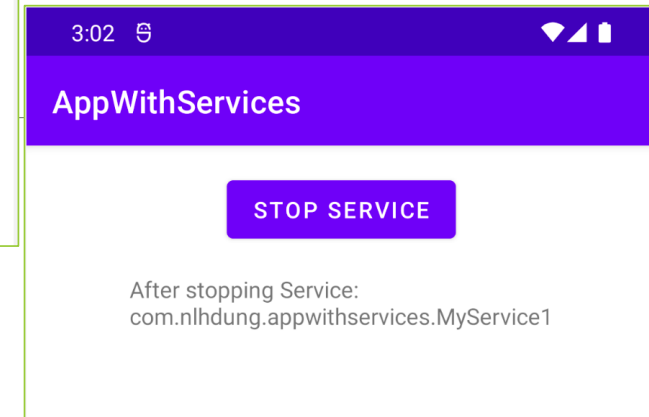
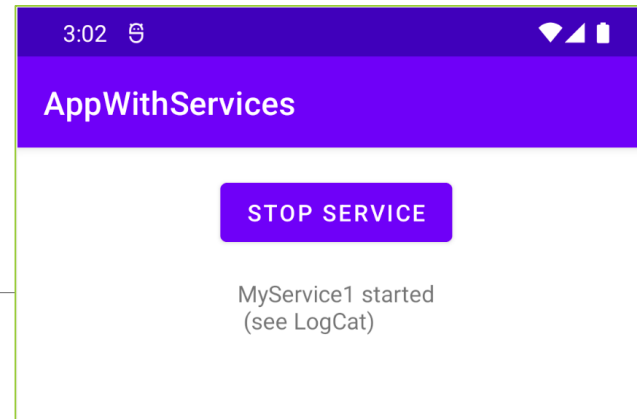
The MyService1

```
class MyService1 : Service() {  
    override fun onBind(p0: Intent?): IBinder? {  
        return null  
    }  
    override fun onStart(intent: Intent?, startId: Int) {  
        super.onStart(intent, startId)  
        Log.d("<<MyService1-onStart>>", "I am alive-1!")  
        Log.d("<<MyService1-onStart>>", "I did something")  
    }  
    override fun onDestroy() {  
        super.onDestroy()  
        Log.d("<<MyService1-onDestroy>>", "I dead")  
    }  
} //MyService1
```

EXAMPLES 1

According to the Log

- 1. Main Activity is started
- 2. Service is started (onCreate, onStart)
- 3. Main Activity UI is displayed
- 4. User stops Service



EXAMPLES 1 (MANIFEST & LAYOUT)

```
<application
  android:allowBackup="true"
  android:icon="@mipmap/ic_launcher"
  android:label="@string/app_name"
  android:roundIcon="@mipmap/ic_launcher_round"
  android:supportRtl="true"
  android:theme="@style/Theme.AppWithServices">
  <activity
    android:name=".TestMyService1"
    android:exported="true">
    <intent-filter>
      <action android:name="android.intent.action.MAIN" />
      <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
  </activity>
  <service android:name="MyService1" />
</application>
```

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".TestMyService1">

  <TextView
    android:id="@+id/txtMsg"
    android:layout_width="wrap_content" android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/btnStopService" />

  <Button
    android:id="@+id/btnStopService"
    android:layout_width="wrap_content" android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:text="Stop Service"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

EXAMPLES 2

(A more interesting activity-service interaction)

1. The main activity starts the service and registers a receiver.
 2. The service is slow; therefore it runs in a parallel thread its time consuming task.
 3. When done with a computing cycle, the service adds a message to an intent.
 4. The intent is broadcasted using the filter: `matos.action.GOSERVICE3`.
 5. A `BroadcastReceiver` (defined inside the main Activity) uses the previous filter and catches the message (displays the contents on the main UI).
 6. At some point the main activity stops the service and finishes executing.
- 

EXAMPLES 2

(A more interesting activity-service interaction)

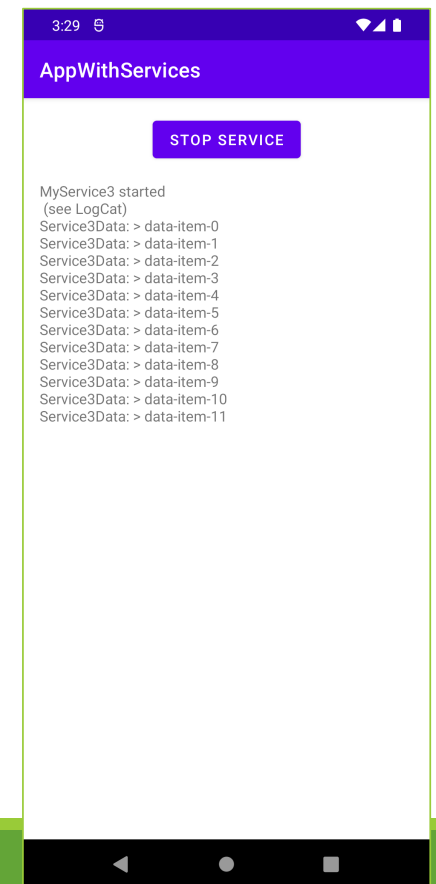
```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MyServiceDriver3">
<EditText
    android:id="@+id/txtMsg"
    android:layout_width="0dp" android:layout_height="0dp"
    android:layout_marginTop="16dp" android:layout_marginBottom="16dp"
    android:clickable="false" android:focusable="false"
    android:gravity="start|top" android:inputType="textMultiLine"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/btnStopService" />
```

```
<Button
    android:id="@+id/btnStopService"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:text="Stop Service"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

EXAMPLES 2

(A more interesting activity-service interaction)

```
<?xml version="1.0" encoding="utf-8"?>
...
<activity
  android:name=".MyServiceDriver3"
  android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>
<service android:name="MyService3" />
...
```



EXAMPLES 2

(A more interesting activity-service interaction)

```
class MyServiceDriver3 : AppCompatActivity() {  
    lateinit var intentMyService1 : Intent  
    lateinit var service: ComponentName  
    var receiver: BroadcastReceiver? = null  
    lateinit var txtMsg: TextView  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
        txtMsg = findViewById(R.id.txtMsg);  
        txtMsg.setText("MyService3 started\n (see LogCat)")  
  
        intentMyService1 = Intent(this, MyService3::class.java)  
        service = startService(intentMyService1)!!  
        val mainFilter = IntentFilter("GOSERVICE3")  
        receiver = MyMainLocalReceiver(txtMsg)  
        registerReceiver(receiver, mainFilter)  
    }  
}
```

```
val btnStopService : Button = findViewById(R.id.btnStopService)  
btnStopService.setOnClickListener {  
    stopService(intentMyService1)  
    unregisterReceiver(receiver)  
    txtMsg.setText("After stopping Service: \n" + service.getClassName())  
}  
}
```

EXAMPLES 2

(A more interesting activity-service interaction)

```
override fun onDestroy() {
    super.onDestroy()
    stopService(intentMyService1)
    unregisterReceiver(receiver)
}

class MyMainLocalReceiver(var txtMsg: TextView) : BroadcastReceiver() {
    override fun onReceive(p0: Context?, p1: Intent?) {
        val serviceData = p1?.getStringExtra("service3Data");
        Log.d ("MyService3 MAIN>>>", "Data received from Service3:" + serviceData);
        val now = "\nService3Data: > " + serviceData
        txtMsg.append(now);
    }
}
```

EXAMPLES 2

(A more interesting activity-service interaction)

```
class MyService3 : Service() {
    var isRunning = true

    override fun onStart(intent: Intent?, startId: Int) {
        super.onStart(intent, startId)
        Log.d("<<MyService3-onStart>>", "I am alive-3!")
        Log.d("<<MyService2-onStart>>", "I did something")
        val serviceThread = Thread(Runnable {
            var i = 0
            do {
                Thread.sleep(100);
                val intentDataForMyClient = Intent("GOSERVICE3")
                val msg = "data-item-" + i ; i+=1
                intentDataForMyClient.putExtra("service3Data", msg)
                sendBroadcast(intentDataForMyClient)
            }while (i<120 && isRunning)
        })
        serviceThread.start();
    }
}
```

```
class MyService3 : Service() {

    override fun onBind(p0: Intent?): IBinder? {
        return null
    }

    override fun onDestroy() {
        super.onDestroy()
        isRunning = false
        Log.d("<<MyService3-onDestroy>>", "I dead-3")
    }
}
```

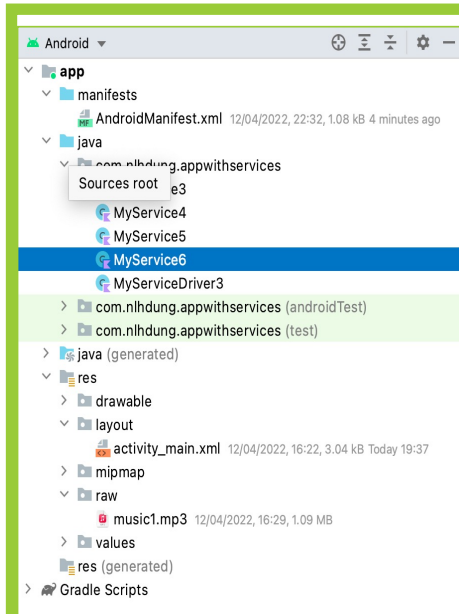
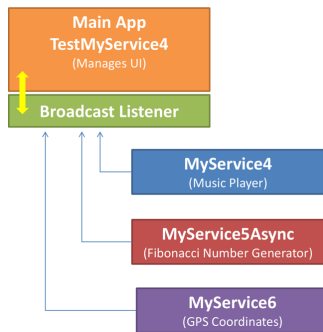
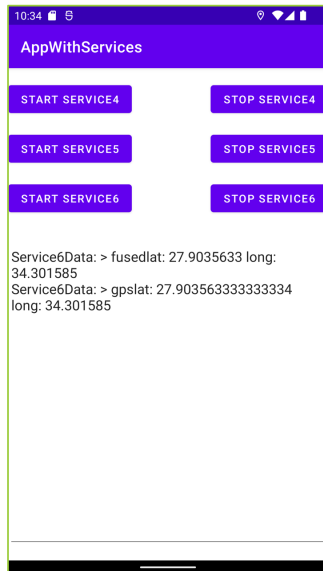
EXAMPLES 3

(An app connected to multiple services)

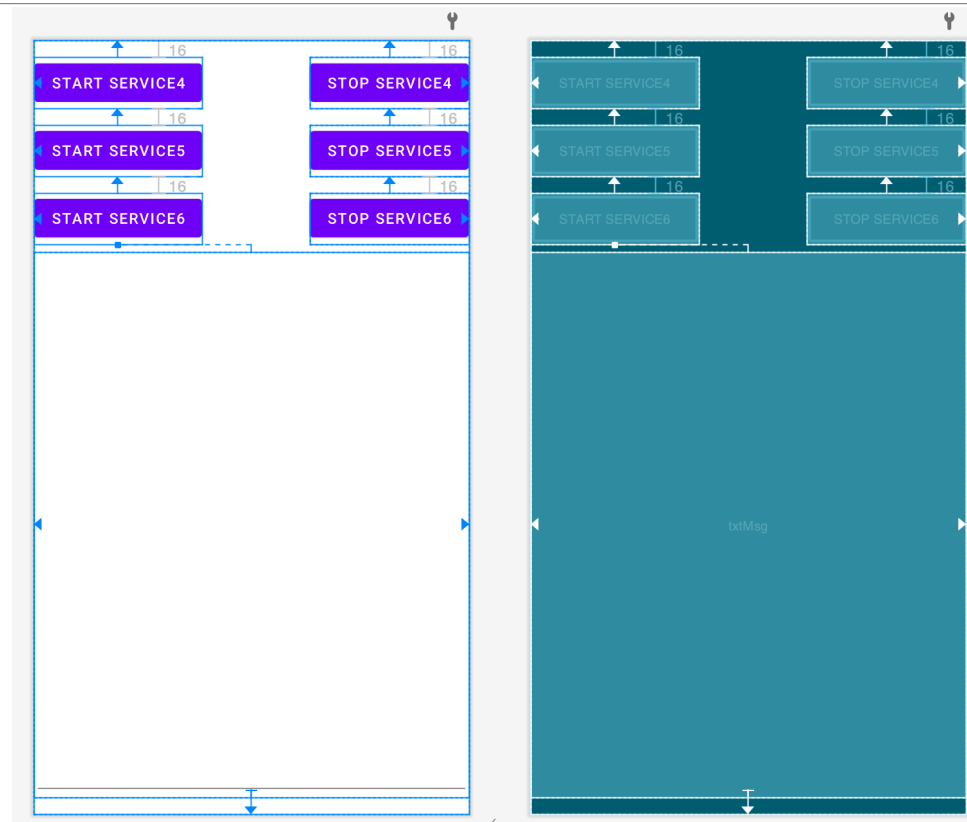
In this application the Main Activity starts 3 services:

1. MyService4: A music player whose input is an mp3 resource file stored in res/raw.
2. MyService5: A service producing Fibonacci numbers in the 20-50 range. The task of number generation is implemented inside an AsyncTask. The efficiency of this Fibonacci implementation is $O(2^n)$ [intentionally slow!]
3. MyService6: The service returns GPS coordinates. Two methods are used to obtain the current location (a) a quick Network-provider based reading (coarse location) , and (b) a more precise but slower Satellite reading (fine location).

The Main Application defines and registers a BroadcastReceiver capable of attending messages matching any of the 3 filters used by the broadcasting services above. Received results are displayed on the user's screen.



EXAMPLES 3 (LAYOUT)



EXAMPLES 3

(MyService4 – a music player)

```
class MyService4 : Service() {
    var isServiceCreated = false
    var player: MediaPlayer? = null

    override fun onCreate() {
        super.onCreate()
        Toast.makeText(this, "MyService4 Created", Toast.LENGTH_LONG).show();
        Log.d("MyService4", "onCreate");
        isServiceCreated = true;
        player = MediaPlayer.create(getApplicationContext(), R.raw.music1);
    }

    override fun onBind(p0: Intent?): IBinder? { return null }

    override fun onStart(intent: Intent?, startId: Int) {
        super.onStart(intent, startId)
        if(player?.isPlaying ?: true) {
            Toast.makeText(this, "MyService4 Already Started", Toast.LENGTH_LONG).show();
        } else {
            Toast.makeText(this, "MyService4 Started", Toast.LENGTH_LONG).show();
            Log.d("<<MyService4>>", "onStart")
            player?.start()
        }
    }
}
```

```
override fun onDestroy() {
    super.onDestroy()
    Toast.makeText(this, "MyService4 Stopped", Toast.LENGTH_LONG).show();
    Log.d("<<MyService4>>", "onDestroy")
    player?.stop()
    player?.release()
}
}
```


EXAMPLES 3

(MyService5 – a slow Fibonacci number gen)

```
class MyService5 : Service() {
    var isRunning = true

    val handler = object:Handler(){
        override fun handleMessage(msg: Message) {
            super.handleMessage(msg)
            Log.d("MyService5-Handler", "Handler got from MyService5: "
                + msg.obj.toString())
        }
    }
    override fun onBind(p0: Intent?): IBinder? { return null}

    override fun onStart(intent: Intent?, startId: Int) {
        super.onStart(intent, startId)
        Log.d("MyService5-Handler", "I am alive-5")
        // we place slow work of service in an AsyncTask so the response we send
        // our caller who run a "startService(...)" method gets a quick OK from us.
        ComputeFibonacciRecursivelyTask(this).execute(20, 50)
    }
}
```

```
override fun onDestroy() {
    super.onDestroy()
    Toast.makeText(this, "MyService5 Stopped", Toast.LENGTH_LONG).show();
    Log.d("<<MyService5-Handler>>", "onDestroy")
}

public class ComputeFibonacciRecursivelyTask(var service: MyService5) :
    AsyncTask<Int, Int, Void>() {
    fun fibonacci(n: Int): Int {
        return if (n == 0 || n == 1) 1 else fibonacci(n - 1) + fibonacci(n - 2)
    }

    override fun doInBackground(vararg p0: Int?): Void? {
        val start : Int = p0[0] ?: 0
        val end : Int = p0[1] ?: 0
        for (i in start..end) {
            val fibn = fibonacci(i)
            publishProgress(i, fibn)
        }
        return null
    }
}
```

EXAMPLES 3

(MyService5 – a slow Fibonacci number gen)

```
override fun onProgressUpdate(vararg values: Int?) {
    super.onProgressUpdate(*values)
    val intentFilter5 = Intent("GOSSERVICE5")
    val data = "dataItem-5-fibonacci-AsyncTask" + values[0] + ": " + values[1]
    intentFilter5.putExtra("MyService5DataItem", data)

    service.sendBroadcast(intentFilter5)
    // (next id not really needed!!! - we did the broadcasting already)
    val msg = service.handler.obtainMessage(5, data);
    service.handler.sendMessage(msg);
}
} // ComputeFibonacciRecursivelyTask
} // MyService5
```

EXAMPLES 3

(MyService6 – a GPS service broadcasting locations)

```
//A GPS service broadcasting locations
class MyService6 : Service() {
    var isRunning = true
    var serviceThread: Thread? = null
    var lm: LocationManager? = null
    var myLocationListener: GPSListener? = null
    val GPS_FILTER = "GPSFIX"

    override fun onBind(p0: Intent?): IBinder? {
        return null
    }

    override fun onStart(intent: Intent?, startId: Int) {
        super.onStart(intent, startId)
        Log.d("MyGpsService-onStart", "I am alive-GPS")

        serviceThread = Thread(Runnable {
            getGPSFix_Version1(); // uses NETWORK provider
            getGPSFix_Version2(); // uses GPS chip provider
        })
        serviceThread?.start()
    }
}
```

```
fun getGPSFix_Version1()
{
    // Get the location manager
    val locationManager = getSystemService(Context.LOCATION_SERVICE) as LocationManager
    // work with best provider
    val criteria = Criteria(); val provider = locationManager.getBestProvider(criteria, false) ?: ""

    if (ActivityCompat.checkSelfPermission(this,
        Manifest.permission.ACCESS_COARSE_LOCATION) != PackageManager.PERMISSION_GRANTED)
    { return }

    val location = locationManager.getLastKnownLocation(provider)
    if (location != null){
        // capture location data sent by current provider
        val latitude = location.getLatitude(); val longitude = location.getLongitude()
        val myFilteredResponse = Intent(GPS_FILTER);
        myFilteredResponse.putExtra("latitude", latitude)
        myFilteredResponse.putExtra("longitude", longitude)
        myFilteredResponse.putExtra("provider", provider)
        Log.d(">>GPS_Service<<", provider + "lat: " + latitude + " long: " + longitude)
        sendBroadcast(myFilteredResponse) // send the location data out
    }
}
```

EXAMPLES 3

(MyService6 – a GPS service broadcasting locations)

```
fun getGPSFix_Version2()
{
    try {
        Looper.prepare()
        // try to get your GPS location using the LOCATION.SERVIVE provider
        lm = getSystemService(Context.LOCATION_SERVICE) as LocationManager
        // This listener will catch and disseminate location updates
        myLocationListener = GPSListener()
        // define update frequency for GPS readings
        val minTime: Long = 2000 // 2 seconds
        val minDistance = 5f // 5 meter
        // request GPS updates
        if (ActivityCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION) !=
            PackageManager.PERMISSION_GRANTED) { return }
        lm!!.requestLocationUpdates(
            LocationManager.GPS_PROVIDER, minTime, minDistance, myLocationListener!!
        )
        Looper.loop()
    } catch (e: Exception) {
        e.printStackTrace()
    }
}
```

EXAMPLES 3

(MyService6 – a GPS service broadcasting locations)

```
override fun onDestroy() {  
    super.onDestroy()  
    Toast.makeText(this, "MyService5 Stopped", Toast.LENGTH_LONG).show();  
    Log.d("<<MyService5-Handler>>", "onDestroy")  
}  
  
inner class GPSListener : LocationListener {  
    override fun onLocationChanged(p0: Location) {  
        // capture location data sent by current provider  
        val latitude = p0.getLatitude()  
        val longitude = p0.getLongitude()  
        // assemble data bundle to be broadcasted  
        val myFilteredResponse = Intent(GPS_FILTER);  
        myFilteredResponse.putExtra("latitude", latitude)  
        myFilteredResponse.putExtra("longitude", longitude)  
        myFilteredResponse.putExtra("provider", p0.getProvider())  
        Log.d(">>GPS_Service<<", "lat: " + latitude + " long: " + longitude)  
        // send the location data out  
        sendBroadcast(myFilteredResponse);  
    }  
}
```

EXAMPLES 3

(MainActivity: TestMyService4.java)

```
class MyServiceDriver3 : AppCompatActivity() {  
    lateinit var intentMyService4 : Intent  
    lateinit var intentMyService5 : Intent  
    lateinit var intentMyService6 : Intent  
  
    var receiver: BroadcastReceiver? = null  
    lateinit var txtMsg: TextView  
  
    override fun onDestroy() {  
        super.onDestroy()  
        unregisterReceiver(receiver)  
    }  
}
```

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
  
    txtMsg = findViewById(R.id.txtMsg)  
    txtMsg.setText("Main started")  
  
    intentMyService4 = Intent(this, MyService4::class.java)  
    intentMyService5 = Intent(this, MyService5::class.java)  
    intentMyService6 = Intent(this, MyService6::class.java)  
  
    val filter5 = IntentFilter("GOSERVICE5")  
    val filter6 = IntentFilter("GPSFIX")  
    receiver = MyMainLocalReceiver()  
    registerReceiver(receiver, filter5)  
    registerReceiver(receiver, filter6)  
}
```

EXAMPLES 3

(MainActivity: TestMyService4.java)

```
val btnStartService4 : Button = findViewById(R.id.btnStartService4)
btnStartService4.setOnClickListener {
    startService(intentMyService4)
}
val btnStopService4 : Button = findViewById(R.id.btnStopService4)
btnStopService4.setOnClickListener {
    stopService(intentMyService4)
}
val btnStartService5 : Button = findViewById(R.id.btnStartService5)
btnStartService5.setOnClickListener {
    txtMsg.setText("")
    startService(intentMyService5)
}
val btnStopService5 : Button = findViewById(R.id.btnStopService5)
btnStopService5.setOnClickListener {
    stopService(intentMyService5)
}
```

```
val btnStartService6 : Button = findViewById(R.id.btnStartService6)
btnStartService6.setOnClickListener {
    txtMsg.setText("")
    if (ContextCompat.checkSelfPermission(this,
        Manifest.permission.ACCESS_FINE_LOCATION) !=
        PackageManager.PERMISSION_GRANTED ||
        ContextCompat.checkSelfPermission(this,
        Manifest.permission.ACCESS_COARSE_LOCATION) !=
        PackageManager.PERMISSION_GRANTED) {
        ActivityCompat.requestPermissions(this,
            arrayOf(Manifest.permission.ACCESS_FINE_LOCATION,
                Manifest.permission.ACCESS_COARSE_LOCATION), 1)
    } else {
        startService(intentMyService6)
    }
}
val btnStopService6 : Button = findViewById(R.id.btnStopService6)
btnStopService6.setOnClickListener {
    stopService(intentMyService6)
}
}
```

EXAMPLES 3

(MainActivity: TestMyService4.java)

```
override fun onRequestPermissionsResult(requestCode: Int,
    permissions: Array<String>, grantResults: IntArray) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults)
    when (requestCode) {
        1 -> {
            if (grantResults.isNotEmpty() && grantResults[0] ==
                PackageManager.PERMISSION_GRANTED) {
                if ((ContextCompat.checkSelfPermission(this,
                    Manifest.permission.ACCESS_FINE_LOCATION) ==
                    PackageManager.PERMISSION_GRANTED)) {
                    startService(intentMyService6)
                }
            } else {
                Toast.makeText(this, "Permission Denied", Toast.LENGTH_SHORT).show()
            }
            return
        }
    }
}
```


EXAMPLES 3

(MainActivity: TestMyService4.java)

```
inner class MyMainLocalReceiver() : BroadcastReceiver() {
    override fun onReceive(p0: Context?, p1: Intent?) {
        Log.d ("MAIN>>", "ACTION: " + p1?.action.toString());
        if (p1!!.action.equals("GOSERVICE5")) {
            val service5Data = p1!!.getStringExtra("MyService5DataItem")
            Log.d ("MAIN>>", "Data received from Service5: " + service5Data)
            txtMsg.append("\nService5Data: > " + service5Data)
        } else if (p1!!.action.equals("GPSFIX")) {
            val latitude = p1!!.getDoubleExtra("latitude", 0.0)
            val longitude = p1!!.getDoubleExtra("longitude", 0.0)
            val provider = p1!!.getStringExtra("provider")
            val service6Data = provider + "lat: " + latitude.toString() + " long: " + longitude.toString()
            Log.d ("MAIN>>", "Data received from Service5: " + service6Data)
            txtMsg.append("\nService6Data: > " + service6Data)
        }
    }
}
//end MyMainLocalReceiver
} //end MyServiceDriver3
```

EXAMPLES 3 (MANIFEST)

```
<!-- ... -->

<activity
  android:name=".MyServiceDriver3"
  android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>
<service android:name="MyService5" />
<service android:name="MyService4" />
<service android:name="MyService6" />

<!-- ... -->
```